UNIT 3. LIST IN PYTHON

Definition:
A *list* in Python is an ordered, mutable collection of elements enclosed in square brackets `[ ]`.

Example:

```
lst = [10, 20, 30]
```

Features of List

- Ordered collection
- Mutable (can be changed)
- Allows duplicate values
- Can store different data types

Basic Operations

```
lst = [10, 20, 30]

print(lst[0])       # Access
lst.append(40)      # Add
lst.remove(20)      # Remove
len(lst)            # Length
```

List Operations in Detail

1. Accessing Elements

```
lst = [10, 20, 30]
print(lst[1])   # 20
```

---

2. Updating Elements

```
lst[0] = 100
```

---

3. Traversing List

```
for i in lst:
    print(i)
```

---

4. List Methods

```
lst.append(50)
lst.insert(1, 15)
lst.pop()
lst.sort()
```

---

5. Slicing

```
lst = [1, 2, 3, 4, 5]
print(lst[1:4])   # [2,3,4]
```

List in Python

Introduction

A list is one of the most commonly used data structures in Python. It is used to store multiple items in a single variable and allows easy manipulation of data.

Definition

A list is an ordered and mutable collection of elements enclosed in square brackets.

Creating a List

```
lst = [1, 2, 3, 4]
lst2 = ["a", "b", "c"]
lst3 = [1, "Hello", 3.5]
```

Accessing Elements

```
print(lst[0])    # First element
print(lst[-1])   # Last element
```

Updating List

```
lst[1] = 100
```

List Operations

1. Adding Elements

```
lst.append(5)
lst.insert(1, 50)
```

---

2. Removing Elements

```
lst.remove(2)
lst.pop()
```

---

3. Traversing

```
for i in lst:
    print(i)
```

---

4. Slicing

```
print(lst[1:3])
```

List Methods

- `append()` → add element
- `insert()` → insert at position
- `remove()` → delete element
- `pop()` → remove last element
- `sort()` → sort list
- `reverse()` → reverse list

Advantages

- Flexible and dynamic
- Easy to modify
- Stores multiple values

✅ TUPLES IN PYTHON

Definition:
A *tuple* is an ordered, immutable collection of elements enclosed in parentheses `( )`.

Example:

```
t = (10, 20, 30)
```

Features of Tuple

- Ordered collection
- Immutable (cannot be changed after creation)
- Allows duplicate values
- Can store different data types

Basic Operations

```
t = (10, 20, 30)

print(t[0])      # Access
len(t)           # Length
```

Tuple Operations in Detail

1. Creating Tuples

```
t1 = (1, 2, 3)
t2 = ("a", "b", "c")
t3 = (1, "Hello", 3.5)
```

---

2. Accessing Elements

```
print(t1[1])   # 2
print(t1[-1])  # 3
```

---

3. Traversing Tuple

```
for i in t1:
    print(i)
```

---

4. Tuple Operations

```
t = (1, 2, 3)
print(t + (4, 5))   # Concatenation
print(t * 2)        # Repetition
```

---

5. Tuple Methods

```
t = (1, 2, 2, 3)

t.count(2)   # Count occurrences
t.index(3)   # Find index
```

Tuples in Python

Introduction

A tuple is a fundamental data structure in Python used to store multiple elements in a single variable. Unlike lists, tuples are immutable, meaning their elements cannot be modified after creation.

Definition

A tuple is an ordered and immutable collection of elements enclosed in parentheses.

Creating Tuples

```
t1 = (1, 2, 3)
t2 = ("apple", "banana")
t3 = (1, "Hello", 3.5)
```

Accessing Elements

```
print(t1[0])    # First element
print(t1[-1])   # Last element
```

Traversing Tuple

```
for item in t1:
    print(item)
```

Tuple Operations

1. Concatenation

```
t1 = (1, 2)
t2 = (3, 4)
print(t1 + t2)
```

---

2. Repetition

```
print(t1 * 2)
```

---

3. Membership

```
print(2 in t1)
```

Tuple Methods

- `count()` → counts occurrences
- `index()` → returns index

Advantages of Tuple

- Faster than lists
- Immutable (safe data)
- Can be used as keys in dictionaries

Difference Between List and Tuple

List	Tuple
Mutable	Immutable
Uses [ ]	Uses ( )
Slower	Faster

✅ DICTIONARIES IN PYTHON

Definition:
A *dictionary* in Python is an unordered collection of key–value pairs enclosed in `{ }`.

Example:

```
d = {"name": "John", "age": 20}
```

Features of Dictionary

- Stores data as **key–value pairs**
- Keys are **unique**
- Values can be duplicated
- Mutable (can be modified)
- Unordered (no fixed position like index)

Basic Operations

```
d = {"name": "John", "age": 20}

print(d["name"])   # Access
d["age"] = 21      # Update
d["city"] = "Chennai"  # Add
del d["age"]       # Delete
```

Dictionary Operations in Detail

1. Creating Dictionary

```
d1 = {"a": 1, "b": 2}
d2 = dict(x=10, y=20)
```

---

2. Accessing Elements

```
print(d1["a"])
print(d1.get("b"))
```

---

3. Adding & Updating

```
d1["c"] = 3
d1["a"] = 100
```

---

4. Removing Elements

```
d1.pop("b")
d1.clear()
```

---

5. Traversing Dictionary

```
for key in d1:
    print(key, d1[key])
```

---

6. Dictionary Methods

```
d.keys()
d.values()
d.items()
```

Dictionaries in Python

Introduction

A dictionary is a powerful data structure in Python used to store data in the form of key–value pairs. It is widely used for fast data retrieval and representation of structured data.

Definition

A dictionary is a mutable, unordered collection of key–value pairs where each key is unique.

Creating a Dictionary

```
d = {"name": "Alice", "age": 25}
d2 = dict(city="Chennai", country="India")
```

Accessing Elements

```
print(d["name"])
print(d.get("age"))
```

Updating Dictionary

```
d["age"] = 26
```

Adding Elements

```
d["course"] = "Python"
```

Removing Elements

```
del d["age"]
d.pop("name")
d.clear()
```

Traversing Dictionary

```
for key, value in d.items():
    print(key, value)
```

Dictionary Methods

- `keys()` → returns all keys
- `values()` → returns all values
- `items()` → returns key-value pairs
- `update()` → updates dictionary

Advantages

- Fast data access using keys
- Flexible and dynamic
- Stores complex data

Difference Between List and Dictionary

List	Dictionary
Uses index	Uses keys
Ordered	Unordered
[ ]	{ }

✅ FILE OPERATIONS IN PYTHON

Definition:
File operation in Python refers to creating, opening, reading, writing, and closing files to store and retrieve data permanently.

Types of File Operations

1. Open a file
2. Read a file
3. Write to a file
4. Close a file

Opening a File

```
f = open("file.txt", "r")
```

Modes

- `"r"` → Read
- `"w"` → Write
- `"a"` → Append
- `"r+"` → Read & Write

Reading from File

```
f = open("file.txt", "r")
print(f.read())        # Read full content
print(f.readline())    # Read one line
print(f.readlines())   # Read all lines
f.close()
```

Writing to File

```
f = open("file.txt", "w")
f.write("Hello World")
f.close()
```

Appending to File

```
f = open("file.txt", "a")
f.write("New Line")
f.close()
```

Using with Statement (Best Practice)

```
with open("file.txt", "r") as f:
    data = f.read()
```

File Operations in Python

Introduction

File operations in Python allow programs to store data permanently in files. This is useful for saving results, logs, and user data.

Types of Files

1. **Text Files** (.txt)
2. **Binary Files** (.bin, .dat)

Opening a File

```
f = open("sample.txt", "r")
```

File Modes

- `"r"` → Read (default)
- `"w"` → Write (overwrites file)
- `"a"` → Append
- `"x"` → Create new file
- `"rb"` / `"wb"` → Binary mode

Reading a File

```
f = open("sample.txt", "r")

print(f.read())         # Entire file
print(f.readline())     # One line
print(f.readlines())    # List of lines

f.close()
```

Writing to a File

```
f = open("sample.txt", "w")
f.write("Hello Python")
f.close()
```

Appending to a File

```
f = open("sample.txt", "a")
f.write("\nNew Content")
f.close()
```

Closing a File

```
f.close()
```

Using with Statement (Recommended)

```
with open("sample.txt", "r") as f:
    print(f.read())
```

✓ Automatically closes the file
✓ Safer and cleaner

File Methods

- `read()` → reads content
- `readline()` → reads one line
- `write()` → writes data
- `close()` → closes file

Advantages

- Permanent data storage
- Easy data handling
- Supports large data

✅ READING AND WRITING A FILE IN PYTHON

Reading a file: Retrieving data from a file.
Writing a file: Storing data into a file.

Reading a File

```
f = open("file.txt", "r")
data = f.read()
print(data)
f.close()
```

Writing a File

```
f = open("file.txt", "w")
f.write("Hello Python")
f.close()
```

Methods for Reading

```
f = open("file.txt", "r")

print(f.read())        # Reads entire file
print(f.readline())    # Reads one line
print(f.readlines())   # Reads all lines as list

f.close()
```

Writing to File

```
f = open("file.txt", "w")
f.write("First Line\n")
f.write("Second Line")
f.close()
```

Appending to File

```
f = open("file.txt", "a")
f.write("\nNew Line")
f.close()
```

Reading and Writing a File in Python

Introduction

File handling in Python allows programs to store and retrieve data permanently. Reading and writing are the most important file operations.

---

1. Opening a File

```
f = open("sample.txt", "r")
```

Modes

- `"r"` → Read
- `"w"` → Write (overwrites)
- `"a"` → Append
- `"r+"` → Read & Write

---

2. Reading a File

a) read()

Reads entire content.

```
f = open("sample.txt", "r")
print(f.read())
f.close()
```

---

b) readline()

Reads one line at a time.

```
f = open("sample.txt", "r")
print(f.readline())
f.close()
```

---

c) readlines()

Reads all lines as a list.

```
f = open("sample.txt", "r")
print(f.readlines())
f.close()
```

---

3. Writing to a File

```
f = open("sample.txt", "w")
f.write("Hello World")
f.close()
```

✓ Overwrites existing content

---

4. Appending to a File

```
f = open("sample.txt", "a")
f.write("\nNew Data")
f.close()
```

✓ Adds content without deleting existing data

---

5. Using with Statement (Best Practice)

```
with open("sample.txt", "r") as f:
    print(f.read())
```

✓ Automatically closes file
✓ Prevents errors

Advantages

- Permanent data storage
- Easy to read/write data
- Useful in real-world applications

✅ BUILT-IN EXCEPTIONS IN PYTHON

Definition:
Built-in exceptions are predefined errors in Python that occur during program execution and interrupt the normal flow.

Common Built-in Exceptions

- `ZeroDivisionError` → division by zero
- `NameError` → variable not defined
- `TypeError` → wrong data type
- `ValueError` → invalid value
- `IndexError` → index out of range
- `KeyError` → key not found in dictionary
- `FileNotFoundError` → file does not exist

Example

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Handling Built-in Exceptions

1. try and except

```
try:
    num = int("abc")
except ValueError:
    print("Invalid input")
```

---

2. Multiple Exceptions

```
try:
    x = int(input())
    y = 10 / x
except ValueError:
    print("Wrong input")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

---

3. finally Block

```
try:
    f = open("file.txt")
except FileNotFoundError:
    print("File not found")
finally:
    print("Execution completed")
```

Built-in Exceptions in Python

Introduction

Exceptions are errors that occur during the execution of a program. Python provides many built-in exceptions to handle common runtime errors. Handling these exceptions prevents program crashes and ensures smooth execution.

Definition

Built-in exceptions are predefined error types provided by Python that occur during program execution.

Common Built-in Exceptions

Exception	Description
ZeroDivisionError	Division by zero
NameError	Undefined variable
TypeError	Invalid data type
ValueError	Invalid value
IndexError	List index out of range
KeyError	Dictionary key not found
FileNotFoundError	File not found

Exception Handling Mechanism

1. try-except Block

```
try:
    x = int("abc")
except ValueError:
    print("Conversion error")
```

---

2. Multiple except Blocks

```
try:
    x = int(input())
    y = 10 / x
except ValueError:
    print("Invalid input")
except ZeroDivisionError:
    print("Division by zero")
```

---

3. else Block

Executes if no exception occurs.

```
try:
    x = int("10")
except ValueError:
    print("Error")
else:
    print("Success")
```

---

4. finally Block

Always executes.

```
try:
    f = open("file.txt")
except:
    print("Error")
finally:
    print("Done")
```

Advantages of Exception Handling

- Prevents program crash
- Improves reliability
- Helps in debugging
- Maintains normal flow of program